



Movie Revenue Prediction

Project Final Report

Introduction to Artificial Intelligence

Group 26

António Faustino
Maxime Giraudo

9 June 2026

Contents

1	Introduction	1
2	Description of the Dataset	1
3	Proposed Solutions & Algorithms	1
4	Preprocessing & Evolution of Choices	2
5	Results & Model Evaluation	3
5.1	Linear Regression	3
5.2	Random Forest Optimization	3
5.3	Multilayer Perceptron (PyTorch)	3
5.4	Gradient Boosting (XGBoost)	3
5.5	Comparison	4
6	Challenges Encountered	5
7	Discussion & Limits	6
8	Conclusion	7
9	Bibliography	7

List of Figures

1	Relation between budget and revenue	2
---	---	---

List of Tables

1	Final performance metrics on the test set.	4
---	--	---

1 Introduction

The primary objective of this project is to develop a machine learning model capable of predicting the global revenue of movies given a specific set of production attributes such as budget, genres, original language, and release date. We frame this as a supervised regression task – unlike a classification approach that would categorize films into broad success tiers, our goal is to predict a continuous numerical value representing total worldwide box office receipts in US dollars.

Movie revenue prediction is a particularly challenging regression problem for several reasons. First, the distribution of revenues is heavily right-skewed: the vast majority of films earn modest returns, while a small number of blockbusters generate revenues orders of larger magnitude. Second, commercial success depends on factors that are difficult or practically impossible to quantify from production metadata alone – audience reception, marketing spend, release timing relative to competing titles, and cultural context all play significant roles. Third, the available datasets are often incomplete, with large fractions of entries missing budget or revenue figures entirely.

Despite these challenges, building accurate predictive models for film revenue has clear practical value for studios, distributors, and investors at the pre-release stage, where data-driven estimates can inform production and marketing decisions. In this project, we explore four modeling approaches of increasing complexity – Linear Regression, Random Forest, a custom PyTorch Multilayer Perceptron, and XGBoost – evaluating each against a held-out test set and comparing their ability to generalize to unseen films.

2 Description of the Dataset

The data for this project are sourced from Kaggle (*The Movies Dataset*). It provides a comprehensive set of metadata for several thousand films. From the available features, the following were selected as inputs to the model: `budget`, `genres`, `keywords`, `original_language`, `production_companies`, `production_countries`, `release_date`, `runtime`, `spoken_languages`, `cast`, and `director`. These span both numerical variables (e.g., `budget`, `runtime`) and categorical features (e.g., `genres`, `director`). Features such as `vote_average`, `vote_count`, and `popularity` were deliberately excluded, as they are only available after a film’s release and would constitute data leakage. The target variable is the `revenue` column, representing the total global box office receipts.

3 Proposed Solutions & Algorithms

Based on our findings from the midterm stage, we revised our architectural choices to deliver a more robust, non-linear modeling framework. We implemented and compared four models:

Linear Regression: This will serve as our baseline model. It is a simple and interpretable approach that assumes a linear relationship between input features (e.g., `budget`) and the target revenue. It allows us to evaluate the necessity of more complex architectures.

Random Forest: This ensemble learning method is widely used for its ability to handle non-linear dependencies and its robustness against outliers. By combining multiple decision trees, it provides a more accurate estimation than simple linear models.

Deep Multilayer Perceptron (PyTorch Custom): While the midterm version utilized a basic `sklearn` implementation that failed to converge effectively, we developed a standalone, custom neural network using PyTorch for the final pipeline. It features a sequence of dense layers (1024 -> 128 -> 1) equipped with ReLU activations, AdamW optimization and standardizing features through a rigorous normalization process to handle complex tabular structures.

XGBoost Regressor (Gradient Boosting): Added as our advanced state-of-the-art solution for tabular data. It sequentializes weak decision trees using gradient descent principles to optimize predictive capabilities while implementing strict regularization checks to combat overfitting.

4 Preprocessing & Evolution of Choices

The Movies dataset is composed of `credits.csv`, `keywords.csv`, and `movies_metadata.csv`. Our pipeline underwent significant optimization between the midterm phase and the final implementation:

Feature Filtering & String Parsing: Extracted the Director and the first 5 core cast members using literal evaluation (`ast.literal_eval`), ensuring data consistency. Rows missing crucial information were explicitly dropped.

The Budget Imputation Experiment (Before vs. After): During the midterm phase, we proposed two alternative data paths to counter the high volume of incomplete rows (0 values in revenue and budget):

- **Version 1:** Complete filtering out of all entries where `budget = 0`, leading to a clean, smaller subset.
- **Version 2:** Imputing missing values with the median budget to preserve dataset rows.

Final Decision: Experimental evaluation revealed that **Version 2 (imputation) systematically degraded model performance**, introducing unwanted noise that corrupted training gradients. Consequently, we abandoned the imputation strategy and executed strict filtering, working strictly with positive budgets and revenues.

Advanced Encoding Strategies: To handle thousands of unique text variables without suffering from the curse of dimensionality, we filtered text fields to retain only the top N most frequent elements using `collections.Counter`:

- **Genres:** All 20 categories preserved via `MultiLabelBinarizer`.
- **Cast & Crew:** Filtered to the top 150 actors and top 50 directors.
- **Keywords & Production:** Kept the top 100 most frequent keywords and top 50 production companies.

Categorical targets like `original_language` were processed via standard One-Hot Encoding. To finalize features, the `release_date` was parsed into separate numerical `release_month` and `release_year` signals.

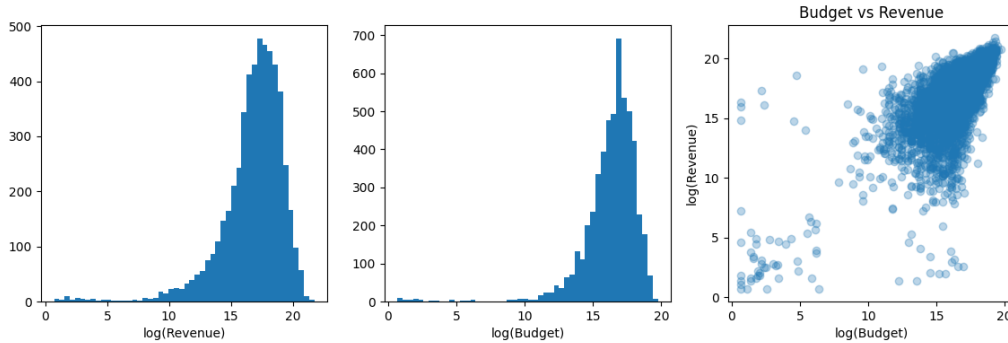


Figure 1: Relation between budget and revenue

5 Results & Model Evaluation

All models were processed through a reproducible 85/15 train-test split (`SEED=42`). The same seed was applied consistently across NumPy, PyTorch, and all scikit-learn and XGBoost models to ensure fully deterministic results across runs. Additionally, all parallelizable models were configured with `n_jobs=1` to guarantee cross-platform reproducibility, as parallel execution can produce non-deterministic floating-point results depending on the operating system and hardware. Performance metrics are tracked in both log-scale and back-transformed absolute USD using a specialized `evaluate()` utility, which also reports the Median Absolute Percentage Error (MdAPE) – a robust alternative to MAPE that is less sensitive to the extreme revenue outliers present in this dataset.

5.1 Linear Regression

Maintained as our structural baseline. It achieved a Test R^2 of **0.1454**, an RMSE of **\$140,654,209**, and a MdAPE of **68.34%**, confirming that simple linear assumptions are highly incapable of translating complex international box-office mechanics.

5.2 Random Forest Optimization

During the midterm, the Random Forest model overfitted severely (Train R^2 : 0.889 vs Test R^2 : 0.460). For the final project, we grid-searched deeper constraints, setting `n_estimators=225`, `max_depth=30`, and `min_samples_split=2`. This strategy successfully stabilized performance, raising the Test R^2 to **0.5160**, lowering the RMSE to **\$105,851,181**, and achieving a MdAPE of **64.34%**, significantly closing the overfitting gap.

5.3 Multilayer Perceptron (PyTorch)

The custom PyTorch MLP was implemented with a three-layer dense architecture ($1024 \rightarrow 128 \rightarrow 1$) using ReLU activations, trained for 1000 epochs with a batch size of 128 and the AdamW optimizer at a learning rate of 0.001. Unlike tree-based methods, the MLP requires feature normalization, as weighted sum computations are sensitive to feature scale; both input features and the target variable were standardized using `StandardScaler` prior to training and inverse-transformed for evaluation. The model achieved a Test R^2 of **0.5612**, an RMSE of **\$100,789,132**, and a MdAPE of **65.57%**, performing comparably to XGBoost on the test set. However, the Train R^2 of **0.9984** reveals severe overfitting, meaning that the network memorized the training data almost perfectly without generalizing proportionally. This is consistent with findings in the literature suggesting that neural networks require significantly larger datasets to learn robust tabular patterns, and that regularization techniques such as dropout or weight decay are essential when training on limited data.

AdamW is a variant of the Adam (*Adaptive Moment Estimation*) optimizer that corrects a subtle flaw in Adam’s implementation of L2 regularization. In standard Adam, weight decay is added directly to the gradient before it is scaled by the adaptive moment estimates — meaning the effective regularization strength varies from parameter to parameter, and is weakest precisely where gradients are largest. AdamW decouples weight decay from the gradient update entirely, applying it directly and uniformly to the weights regardless of gradient history. In practice, this produces more consistent regularization and better generalization, particularly in networks with many parameters. Beyond this correction, AdamW inherits all the advantages of Adam: it uses the first moment (an exponential moving average of gradients, analogous to momentum) to smooth updates across iterations, and the second moment (an exponential moving average of squared gradients, as in RMSProp) to scale the learning rate adaptively per parameter — enabling fast convergence even in the presence of features with very different numerical scales, as is the case in our dataset.

5.4 Gradient Boosting (XGBoost)

XGBoost yielded the best overall results for our tabular dataset. The model was configured with `n_estimators=2000`, `max_depth=9`, `learning_rate=0.12`, `subsample=0.9`, and `colsample_bytree=0.8`. To mitigate overfitting without sacrificing capacity, we used early stopping with a patience of 50 rounds

on a dedicated validation split (15% of the training set), allowing the model to halt training automatically once generalization performance ceased to improve. The final model achieved a Train R^2 of **0.8249**, a Test R^2 of **0.5882**, an RMSE of **\$97,640,658**, and a MdAPE of **63.02%**, making it the best-performing model across all metrics in this project.

It is worth noting that during the hyperparameter search, more aggressive configurations – specifically those with higher `max_depth` values and no early stopping – achieved a higher raw Test R^2 of **0.6120**, but at the cost of complete training saturation, with a Train R^2 of **1.0000**. A model that memorizes the training set perfectly offers no meaningful guarantee of generalization: the apparent gain on the test set is likely the result of overfitting to noise that happens to correlate with test labels, rather than the discovery of genuinely predictive structure. We therefore discarded these configurations in favour of the regularized model, accepting a modest reduction in test performance in exchange for a substantially more reliable and interpretable result.

5.5 Comparison

The evaluation framework combines four complementary metrics. The coefficient of determination (R^2) measures the proportion of variance in the true revenue that the model is able to explain — a value of 1.0 represents a perfect fit, while 0.0 means the model performs no better than predicting the mean revenue for every film. The Root Mean Squared Error (RMSE) expresses the average prediction error in absolute USD, penalising large errors more heavily than small ones due to the squaring operation; it is the most intuitive metric for understanding the practical magnitude of mistakes. The Median Absolute Percentage Error (MdAPE) complements the RMSE by expressing the typical error as a percentage of the true revenue, making it interpretable regardless of the revenue scale – a MdAPE of 63% means that for at least half of the films in the test set, the model’s prediction is within 63% of the actual box office figure. The median is used instead of the mean to avoid distortion from the extreme revenue outliers inherent to the movie industry, where a small number of blockbusters earn orders of magnitude more than the average film.

Table 1 summarizes the performance of the implemented models.

Model Approach	Train R^2	Test R^2	RMSE (USD)	MdAPE
Linear Regression	–	0.1454	\$140,654,209	68.34%
Random Forest	0.9221	0.5160	\$105,851,181	64.34%
Multilayer Perceptron	0.9984	0.5612	\$100,789,132	65.57%
XGBoost	0.8249	0.5882	\$97,640,658	63.02%

Table 1: Final performance metrics on the test set.

6 Challenges Encountered

The development of this pipeline involved a number of technical difficulties, several of which were only identified after unexpected behaviour in intermediate results. This section documents the most significant obstacles encountered throughout the project.

Revenue Zeros as Disguised Missing Values: The dataset contained approximately 36 000 entries with `revenue = 0`. Initially, these were treated as legitimate zero-revenue records. A closer inspection revealed that they were in fact unreported values – films for which box office data simply was not available – stored as zeros rather than `NaN`. Failing to remove them would have severely corrupted the target distribution and biased every model towards predicting near-zero revenues.

Duplicate IDs Across Source Files: The three source files (`movies_metadata.csv`, `credits.csv`, `keywords.csv`) each contained duplicate entries on the `id` column — with `keywords.csv` alone contributing 987 duplicates. Without explicit deduplication before the merge, the resulting dataframe would contain replicated rows, inflating dataset size and leaking repeated samples into both the training and test sets.

Type Mismatch Preventing the Join: The `id` column in `movies_metadata.csv` was stored as a generic Python object (string), while the same column in `credits.csv` and `keywords.csv` was stored as an integer. This type inconsistency caused the `merge` operation to silently produce an empty result. The fix required explicit conversion via `pd.to_numeric()` followed by `.astype(int)`, with non-numeric values coerced to `NaN` and dropped beforehand.

Intermediate List Columns Leaking into the Feature Matrix: During the encoding pipeline, several intermediate columns with the suffix `_list` (e.g., `cast_list`, `keywords_list`) were generated as auxiliary structures. A bug in the column selection logic caused some of these to be included in the final feature matrix alongside their one-hot encoded counterparts. This was resolved by enforcing explicit exclusion of any column ending in `_list` when constructing `cols_to_keep`.

sklearn MLP Producing Pathological Results: The initial MLP implementation used `MLPRegressor` from scikit-learn. This model consistently produced test R^2 values in the range of -867 to -1497 , indicating that it was performing far worse than a trivial mean predictor. The instability arose from human error, combination of unscaled inputs, an inappropriate architecture, and the absence of target normalization. Replacing the sklearn MLP with a custom PyTorch implementation – with proper `StandardScaler` applied to both features and target – resolved the issue entirely.

Necessity of Scaling the Target Variable in the MLP: A less obvious requirement of the PyTorch MLP was that the target variable (`revenue_log`) also needed to be standardized before training, not just the input features. Without scaling the target, the raw log-revenue values (typically in the range 12–23) caused the MSE loss to remain numerically large, producing unstable gradients and preventing convergence. Only after applying `StandardScaler` to `y_train` and inverse-transforming predictions at evaluation time did the MLP begin to converge reliably.

Data Leakage in Initial Categorical Encoding: The first encoding strategy for `director` and `cast` used target encoding – replacing each category with the mean `revenue_log` of films associated with it. Since this mean was computed over the full dataset before the train-test split, information from the test set leaked into the training process. The encoding was replaced with one-hot encoding over the top- N most frequent values, which is computed purely from the training data and does not reference the target.

Random Forest Overfitting with No Winning Regularization: The Random Forest exhibited strong overfitting (Train $R^2 \approx 0.92$ vs Test $R^2 \approx 0.52$). Attempts to regularize by reducing `max_depth` consistently worsened test performance rather than improving it, suggesting that the model genuinely required deep trees to capture signal in the sparse, high-dimensional feature space. The final configuration retained `max_depth=30` as the best achievable trade-off given the dataset size.

7 Discussion & Limits

The final implementation successfully demonstrated the superiority of gradient-boosted tree architectures over linear baselines for complex tabular datasets. However, the development cycle also exposed several structural limits that constrain the models’ predictive ceiling:

- **Data Quality over Data Quantity:** Our initial strategy during the midterm hoped to preserve dataset size through median budget imputation. Our final experiments proved that domain-specific tabular models are highly sensitive to artificial noise. In movie revenue prediction, structural zeros (unreported budgets) cannot be treated as statistical averages; strict filtering, though reducing dataset size, yields far more reliable gradients.
- **The Feature Engineering Ceiling:** While our frequency-capped `MultiLabelBinarizer` allowed the models to ingest high-dimensional categorical metadata (directors, top cast, keywords), binary flags are a coarse representation. They encode the presence or absence of a name, but carry no information about the commercial track record, cultural relevance, or audience reach of a given director or actor at the time of a film’s release. This is a fundamental limitation of the encoding strategy and likely accounts for a significant portion of the unexplained variance in our predictions.
- **Advanced Regularization Protocols:** As observed in our XGBoost experiments, high-capacity models easily achieve a perfect 1.0000 R^2 training score by memorizing specific production combinations. The use of early stopping on a dedicated validation split proved effective in preventing this collapse, though it required accepting a modest reduction in raw test performance in exchange for a more reliable and generalizable result.

8 Conclusion

Predicting global movie revenue remains an intrinsically volatile challenge due to shifting cultural trends, viral marketing, and unquantifiable consumer behaviour. Throughout this project, we successfully transitioned from a heavily overfitted midterm baseline to a more robust and regularized machine learning framework, progressing through four models of increasing complexity and expressiveness.

The Linear Regression baseline confirmed that the relationship between production features and box office revenue is fundamentally non-linear, achieving a Test R^2 of only **0.1454**. The Random Forest improved substantially on this, reaching a Test R^2 of **0.5160**, though it exhibited persistent overfitting that proved resistant to regularization — a consequence of the dataset’s high dimensionality and sparsity, which required deep trees to extract meaningful signal. The PyTorch MLP demonstrated that neural networks can be competitive on tabular data when training is properly instrumented: correct scaling of both inputs and target variable, the AdamW optimizer, and careful seed management were all necessary conditions for convergence, ultimately yielding a Test R^2 of **0.5612**. Finally, the XGBoost Regressor, trained with early stopping on a dedicated validation split, achieved the best overall result with a Test R^2 of **0.5882** and an RMSE of **\$97,640,658**.

Beyond the modelling results, this project underscored the critical importance of rigorous data preprocessing. The removal of disguised missing values, the resolution of ID type mismatches, the deduplication of source files, and the elimination of data leakage in categorical encoding were each decisive steps without which no model would have produced reliable results. In real-world financial forecasting, the discipline applied to the data pipeline is at least as consequential as the choice of algorithm.

9 Bibliography

References

- [1] Scikit-learn. Scikit API Reference. [online] Available at: <https://scikit-learn.org/stable/api/index.html>
- [2] Banik, R. (n.d.). The Movies Dataset. [online] [www.kaggle.com](https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset/data). Available at: <https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset/data>.
- [3] Pytorch.org. (n.d.). Build the Neural Network — PyTorch Tutorials 2.7.0+cu126 documentation. [online] Available at: https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html.
- [4] XGBoost Parameters — xgboost 3.2.0 documentation. [online] Available at: https://xgboost.readthedocs.io/en/release_3.2.0/parameter.html.
- [5] Lecture Materials.